

Raport z projektu: Etap 4

Zastosowanie Fine-Tuningu i wektorowych baz danych (FAISS) do rekomendacji gier

Robert Suwarski, Dominik Sawicki, Mateusz Wojno, Rafał Duszyński

7 czerwca 2026

1 Cel aktualizacji (Etap 4)

Głównym celem czwartego etapu projektu było ulepszenie odpowiedzi po polsku, zaimplementowano proces douczania (fine-tuning). Dodatkowo usprawniono system RAG (Retrieval-Augmented Generation) zaimplementowanego w Etapie 3. Zdecydowano się na odejście od prostej wektoryzacji opartej na częstotliwości słów (TF-IDF) na rzecz wielowymiarowych embeddingów semantycznych.

System w obecnej architekturze:

- Generuje wektory semantyczne przy użyciu zaawansowanego, wielojęzycznego modelu transformującego.
- Wykorzystuje bazę FAISS do błyskawicznego odnajdywania podobnych gier w przestrzeni wektorowej.
- Wykorzystuje mechanizm *re-rankingu*, łączący trafność semantyczną z realnym odbiorem gier (odsetek pozytywnych recenzji).
- Formułuje rekomendacje przy użyciu sfinetunowanego modelu LLM dopasowanego do naszych potrzeb.
- [Github](#)

2 Wykorzystane modele i proces trenowania

W projekcie zachowano bazowy model **Qwen/Qwen2.5-1.5B-Instruct**. Został on jednak udoskonalony za pomocą techniki QLoRA, nakładając specjalny "plaster wiedzy". Trening odbył się z wykorzystaniem pliku jsonl z przykładami zapytań i odpowiedzi. (techniką few-shot prompting) Trening miał za zadanie zmienić charakter/ton odpowiedzi modelu Qwen na bardziej przyjazny, "kumpelski".

Trening przeprowadzono z zachowaniem rygorystycznych ograniczeń sprzętowych, celując w działanie na kartach graficznych o pojemności 6GB VRAM (w tym przypadku NVidia GTX 1660 Ti). Użyto optymalizacji:

- **Kwantyzacja 4-bitowa:** Parametry w `BitsAndBytesConfig` (NF4, Double Quant, precyzja obliczeniowa FP16) drastycznie zmniejszyły zużycie pamięci.

- **Parametry LoRA:** Zastosowano range $r = 8$ oraz $\alpha = 16$.
- **Optymalizator i Batching:** Ustawiono `batch_size=1` z symulacją poprzez `gradient_accumulation_steps=4` oraz wykorzystano optymalizator `paged_adamw_32bit`.

Wyuczony adapter zapisano lokalnie i jest on dynamicznie wczytywany (naklejany) na bazowy model podczas startu aplikacji pomocniczej.

3 Nowa architektura wyszukiwania (RAG)

Zamiast korzystać z TF-IDF, wprowadzono nowy potok przetwarzania danych:

1. **Embeddingi:** Gra wektoryzowana jest na etapie ładowania przy użyciu wielojęzycznego modelu `paraphrase-multilingual-MiniLM-L12-v2`, który kompresuje sklejony opis, tagi i kategorie do wektora o 384 wymiarach.
2. **Baza FAISS:** Zbudowano indeks `IndexFlatL2`, który dla 100 000 wpisów błyskawicznie przeszukuje przestrzeń L2. Indeks jest zapisywany na dysk (`games_faiss.index`), dzięki czemu kolejne uruchomienia są niemal natychmiastowe.
3. **Over-fetching i Re-ranking:** System pobiera początkowo 500 rekordów najbliższych semantycznie. Następnie oblicza *final score* oparty w 50% na znormalizowanej odległości FAISS i w 50% na znormalizowanym wskaźniku pozytywnych opinii (**Positive**). Rozwiązuje to problem „gier o wybitnych opisach, ale tragicznej jakości”. Dopiero wyselekcjonowane w ten sposób Top-5 trafia do LLM.

Zabezpieczenie przed halucynacjami i niską jakością

Dzięki wprowadzonym zmianom zminimalizowano sytuację, w której model poleca niszową, słabo ocenianą grę tylko ze względu na dobrze pasujące słowa kluczowe z zapytania użytkownika. Przesunięcie tagów oraz gatunków na przód opisów pozwala embeddingom łatwiej wyłapać najważniejsze cechy tytułu, a ostateczny re-ranking gwarantuje rekomendację gier polecanych przez samych graczy.

4 Zasoby i ograniczenia

- **Przestrzeń i RAM:** Przeliczenie 100 000 opisów wymaga jednorazowego obciążenia GPU, jednak po zbudowaniu pliku indeksu operacje wyszukiwania zajmują ułamek sekundy i odciążają CPU/RAM z konieczności przechowywania w pamięci ciężkiej macierzy TF-IDF, używanej w Etapie 3.
- **Sprzęt:** Dzięki 4-bitowej kompresji oraz ładowaniu gotowego adaptera model potrafi wydajnie generować rekomendacje nawet na słabszym sprzęcie komputerowym z ograniczoną pamięcią VRAM.
- **Ograniczenie wolumenu danych:** Ze względu na restrykcyjne limity pamięci VRAM oraz potrzebę sprawnej weryfikacji architektury QLoRA, proces douczania (fine-tuning) został przeprowadzony na mocno okrojonym zbiorze zawierającym

jedynie 30 przykładowych rekordów z pliku `dane_treningowe.jsonl`. Przy komercyjnych projektach zbiór danych treningowych jest znacznie, ale to znacznie większy.

5 Informacja o tym jak uruchomić aplikację

5.1 Wymagania wstępne

Przed uruchomieniem któregośkolwiek ze skryptów należy upewnić się, że w środowisku zainstalowane są odpowiednie biblioteki oraz że dostępne jest środowisko wykonawcze z obsługą CUDA (karta graficzna NVIDIA), ponieważ oba procesy ściśle wymagają akceleracji GPU do poprawnego i wydajnego działania. Wymagane pakiety można zainstalować w terminalu za pomocą menedżera pip:

```
pip install pandas numpy torch transformers peft trl datasets bitsandbytes  
faiss-gpu sentence-transformers kagglehub
```

5.2 Krok 1: Przygotowanie i uruchomienie procesu uczenia (training.py)

Skrypt `training.py` odpowiada za przeprowadzenie procesu fine-tuningu (QLoRA) na bazie modelu Qwen/Qwen2.5-1.5B-Instruct. Procedura uruchomienia wygląda następująco:

1. Należy upewnić się, że plik z przygotowanymi danymi treningowymi o nazwie `dane_treningowe.jsonl` znajduje się w tym samym katalogu co skrypt `training.py`.
2. Uruchomić skrypt z poziomu terminala za pomocą polecenia:

```
python -X utf8 training.py
```

3. Skrypt automatycznie pobierze model bazowy, zastosuje 4-bitową kompresję w celu zaoszczędzenia pamięci VRAM i rozpocznie proces treningu LoRA.
4. Po zakończeniu domyślnych 100 kroków uczenia (`max_steps`), sfinetunowane wagi (adapter) oraz zaktualizowany tokenizer zostaną automatycznie zapisane w nowo utworzonym folderze `./qwen_gotowy`.

5.3 Krok 2: Uruchomienie głównej aplikacji rekomendującej (main.ipynb)

Główna aplikacja integruje bazę wektorową FAISS oraz douczony w poprzednim kroku model do obsługi interaktywnych zapytań użytkownika. Instrukcja uruchomienia:

1. Należy upewnić się, że w katalogu roboczym aplikacji znajdują się:
 - [Plik](#) z bazą gier `games.csv` zawierający dane do przeszukiwania.
 - Folder `./qwen_gotowy` (wygenerowany w kroku treningowym) zawierający nauczane wagi adaptera.
2. Otworzyć plik `main.ipynb` w środowisku obsługującym Jupyter Notebooks (tutaj użyto PyCharm) i uruchomić kolejno wszystkie komórki (lub alternatywnie wyeksportować kod do zwykłego pliku `.py` i wywołać go w terminalu).

3. **Zachowanie przy pierwszym uruchomieniu:** Jeśli plik indeksu `games_faiss.index` nie zostanie znaleziony na dysku, aplikacja pobierze model embeddingowy `paraphrase-multilingual-MiniLM-L12-v2` i rozpocznie transformację opisów gier z pliku CSV do postaci wektorowej. Po zakończeniu tworzenia bazy FAISS, indeks zostanie zapisany na dysku do ponownego użycia.
4. **Zachowanie przy kolejnych uruchomieniach:** Jeśli plik `games_faiss.index` już istnieje, system pominie proces wektoryzacji i wczyta gotową bazę w ułamku sekundy.
5. Po pomyślnym załadowaniu indeksu i naklejeniu adaptera na model Qwen, w konsoli uruchomi się interaktywna pętla tekstowa:

Napisz, jakiej gry szukasz albo wpisz 'exit':

6. Wpisanie polecenia do szukania gry i naciśnięcie klawisza Enter spowoduje wyświetlenie 5 najbliższych dopasowań z bazy danych wraz z uzasadnieniem sfinetunowanego modelu. Wpisanie komendy `exit`, `quit` lub `koniec` bezpiecznie zamyka aplikację.

6 Przykładowe uruchomienie aplikacji (Log z terminala)

Poniżej znajduje się zrzut logów z terminala podczas testowego uruchomienia zaktualizowanej aplikacji. Zapytano model "Poleć mi grę RPG".

```
Ładowanie modelu embeddingowego...
Znaleziono zapisany indeks! Wczytywanie z games_faiss.index...
Wczytano błyskawicznie!
Czy GPU jest dostępne? True
Ładowanie bazy...
Naklejanie plastra LoRA...
```

Napisz, jakiej gry szukasz albo wpisz 'exit':

Najbardziej podobne gry z CSV:

	Name	distance
4193	Dota 2	12.398281
8878	PUBG: BATTLEGROUNDS	10.815097
54613	OCTOPATH TRAVELER II	7.833295
92942	Your Chronicle	8.143900
59354	Baldur's Gate 3	10.619790

Odpowiedź modelu:

Niektóre możliwości w twoim wyborze są bardzo podobne, ale jeśli chciałbyś znaleźć pełny rozwiązację, nie mogę się skupić na jednej gnieńce.

Ta gra ma dużo wspólnego z PUBG: Battlegrounds, która jest jedną z najpopularniejszych gier w świecie.

Zamiast tego sugeruję Twojemu osobiście odtworzyć Octopath Traveler II lub Baldur's Gate 3. Zabawne zabawa i straży nie przerywają.

Ale jeśli cię interesują RPG, takie jak Dragon Age Origins czy Mass Effect, możesz również wybrać tę gwarancję, która będzie idealna dla Twojej godzin pracy.

Jeden z moich rekordów wynikający od tej gry był 1036. Niechbędzie cię nie zaskoczy.

Ocena poprawionego wnioskowania

Widoczna na zrzucie hierarchia odległości pokazuje poprawne działanie **re-rankingu**. Sam sfinetunowany model używa wygenerowanego kontekstu oraz precyzuje swoją odpowiedź, starając się lepiej dostosować ton do gracza. Interesujące jest też to, że model podjął decyzję, żeby polecić graczowi Octopath Traveler II oraz Baldur's Gate 3 kosztem Doty 2.